NL-FDEP

Software Requirements Specification

Natural Language to Flow Diagram Encoder Pipeline

Student ID	**24038155**
Version	4.37
System baseline	NL-FDEP v4.37 study configuration
Repository	https://github.com/ajohnson-2210/Final_Project

Purpose

This Software Requirements Specification (SRS) defines the as built requirements of the Natural Language to Flow Diagram Encoder Pipeline (NL-FDEP). It describes the behaviour required of the processing pipeline, validation controls, artefact retention mechanisms, behavioural evaluation, Human Review support and downstream analytical dashboards used in the study configuration.

The specification is intended to support reproducibility, repository documentation, maintenance and future extension. It captures the operational requirements demonstrated by the v4.37 study baseline while separating those requirements from future research proposals such as a generalised Controlled Intermediate Language (CIL).

Scope

The NL-FDEP is a multi-stage transformation and evaluation system for structured algorithmic representations. Its principal experimental pathway begins with pseudocode generated by an external transformer based language model and progresses through canonicalisation, deterministic parsing, Abstract Syntax Tree construction, machine readable control flow representation, deterministic Python generation, behavioural testing and analytical reporting.

The principal processing sequence is:

Pseudocode → Processed Pseudocode → AST → Flow Representation → Python → Test Comparative Results

The NL-FDEP also supports a structured Draw.io flow diagram as an alternative entry point into the graph to Python and behavioural testing stages. This alternative pathway is a supported capability but is not the principal input route used for the six-model comparative experiment.

System Context and Boundaries

Area	NL-FDEP responsibility	Outside NL-FDEP responsibility
Natural language task	Maintain the study specification as the common experimental input and retain its association with generated artefacts.	Generate the model response or control the internal reasoning of external language models.
Pseudocode	Accept, retain, canonicalised and parse model generated pseudocode under the defined processing profile.	Repair missing algorithmic logic or invent conditions, loops or operations absent from the model output.
Intermediate representations	Generate and retain AST and machine readable flow representations and provide visual views.	Treat visual layout, colour or connector routing as program semantics.
Executable representation	Generate Python deterministically from the validated flow representation and compile/execute it.	Generate Python directly from the original pseudocode in the principal pathway.
Evaluation	Run frozen behavioural tests, record rubric evidence, support Human Review and generate study summaries.	Treat passing tests as formal proof of universal semantic equivalence.
Reporting	Produce read-only analytical dashboards and exports from retained study artefacts.	Alter raw experimental outcomes retrospectively through dashboard interaction.

Normative Language

Term	Meaning
SHALL	Mandatory requirement for the defined system baseline.
SHOULD	Recommended requirement whose omission requires a documented justification.
MAY	Permitted optional capability.

Stakeholders

- Researcher/developer: configures, executes, validates and maintains NL-FDEP.
- Human reviewer: examines retained artefacts and records rubric based assessments independently of automated scoring.
- Research reader/examiner: inspects evidence, methodology, traceability and study outputs.
- Future maintainer: updates processing rules, tests or representations while preserving versioned behaviour and regression evidence.

Design Constraints and Assumptions

ID	Constraint
DC-01	The principal experimental input to NL-FDEP is model generated pseudocode rather than natural language text itself.
DC-02	The pseudocode interface is deliberately constrained to support deterministic machine interpretation.
DC-03	The strict pseudocode conventions are adapted from the Cambridge International AS & A Level Computer Science 9618 Pseudocode Guide for Teachers and are further restricted by NL-FDEP-specific grammar and processing rules.
DC-04	Canonicalisation may normalise supported representation differences but shall not invent algorithmic meaning.
DC-05	The flow representation is a genuine computational stage: generated Python shall depend on the validated graph rather than bypassing it.
DC-06	Python 3 is the executable target language for the implemented study configuration.
DC-07	The 50 algorithm, six model study profile is an experimental configuration and shall not be treated as an intrinsic hard limit of the architecture.
DC-08	The trusted baseline is a study readiness control and shall not operate as a runtime oracle for model correctness.
DC-09	Dashboards are analytical/presentation components and shall not modify authoritative raw study artefacts or scoring evidence.
DC-10	CIL is a future/generalised research concept; the v4.37 strict pseudocode dialect shall not be represented as a universal CIL standard.

Functional Requirements

The following requirements are grouped by processing responsibility. Requirement identifiers are stable references intended for implementation traceability, testing and issue tracking.

Input and Experimental Configuration

ID	Requirement	Priority	Verification
FR-IN-01	The system SHALL accept a model generated pseudocode artefact as the principal processing input for each algorithm instance.	Must	Inspection / test
FR-IN-02	The system SHALL associate each input artefact with a unique algorithm identifier and dataset/model identifier.	Must	Data validation
FR-IN-03	The system SHALL preserve the original model generated pseudocode without destructive modification.	Must	Artefact inspection
FR-IN-04	The system SHALL support a configured algorithm set rather than relying on a fixed model specific task list in code.	Must	Configuration test
FR-IN-05	The study configuration SHALL support the complete 50 algorithm benchmark for each evaluated model dataset.	Must	Dataset consistency check
FR-IN-06	The system SHALL retain the relationship between each natural language specification, model generated pseudocode and downstream artefacts.	Must	Traceability inspection
FR-IN-07	The system SHOULD support additional model datasets without requiring changes to the transformation semantics.	Should	Extension test

Pseudocode Processing and Canonicalisation

ID	Requirement	Priority	Verification
FR-CAN-01	The system SHALL canonicalise accepted pseudocode according to a named/versioned processing profile before deterministic parsing.	Must	Unit / regression test
FR-CAN-02	Canonicalisation SHALL be limited to recognised representational transformations and SHALL NOT invent missing algorithmic operations, branches, conditions, loops or returns.	Must	Rule inspection / negative test
FR-CAN-03	The system SHALL retain the processed pseudocode separately from the original pseudocode.	Must	Artefact inspection
FR-CAN-04	The system SHALL produce a processing report describing transformations applied during canonicalisation.	Must	Report inspection
FR-CAN-05	For the same input, processing profile and software version, canonicalisation SHALL produce reproducible output.	Must	Repeatability test
FR-CAN-06	Unsupported or ambiguous forms that cannot be normalised safely SHALL be surfaced as explicit processing or parsing failures rather than silently inferred.	Must	Negative test
FR-CAN-07	The system SHOULD retain sufficient processing metadata to permit later analysis of canonicalisation burden by dataset or algorithm.	Should	Artefact inspection

Parsing and Abstract Syntax Tree

ID	Requirement	Priority	Verification
FR-AST-01	The parser SHALL convert accepted processed pseudocode into a machine readable Abstract Syntax Tree (AST).	Must	Parser test
FR-AST-02	Parser behaviour SHALL be deterministic for a fixed accepted input and pipeline version.	Must	Repeatability test
FR-AST-03	The parser SHALL use the defined NL-FDEP grammar and structural rules rather than probabilistic interpretation.	Must	Implementation review
FR-AST-04	The system SHALL retain the machine readable AST as a first class study artefact.	Must	Artefact inspection
FR-AST-05	The system SHOULD provide a visual AST representation for inspection without treating that visual representation as computational authority.	Should	UI / artefact inspection
FR-AST-06	Parser failures SHALL be reported at the parsing stage and SHALL not be misclassified as downstream flow or Python failures.	Must	Failure-localisation test

Flow Representation and Visualisation

ID	Requirement	Priority	Verification
FR-FLOW-01	The system SHALL transform the AST into a machine readable graph representing execution/control flow relationships.	Must	Transformation test
FR-FLOW-02	The machine readable graph SHALL explicitly represent sequences, decisions, branches, loops and termination relationships where present.	Must	Graph inspection / unit test
FR-FLOW-03	The machine readable graph SHALL be the authoritative flow artefact for the principal pseudocode input pathway.	Must	Implementation review
FR-FLOW-04	The system SHALL validate the graph before downstream Python generation.	Must	Validation test
FR-FLOW-05	Invalid graph structures SHALL produce an explicit flow-validation failure and SHALL not proceed silently to Python generation.	Must	Negative test
FR-FLOW-06	The system SHALL generate a human-readable flow visualisation from the machine-readable graph.	Must	Visual inspection
FR-FLOW-07	The system SHALL support export of a Draw.io compatible flow diagram representation.	Must	Export/import test

ID	Requirement	Priority	Verification
FR-FLOW-08	Graphical position, colour and connector routing SHALL NOT determine execution semantics in the principal pathway.	Must	Implementation review

Alternative Structured Flow Input

ID	Requirement	Priority	Verification
FR-ALT-01	The system SHALL support a structured Draw.io flow diagram as an alternative input route.	Must	End-to-end test
FR-ALT-02	Imported flow input SHALL be converted into the machine readable graph representation and validated before Python generation.	Must	Import/validation test
FR-ALT-03	The alternative flow input route SHALL enter the normal downstream Python generation and behavioural testing stages rather than using a separate execution semantics.	Must	End-to-end test
FR-ALT-04	Use of the alternative flow-input route SHALL be distinguishable in retained metadata from the principal pseudocode-input route.	Must	Artefact inspection

Python Generation and Compilation

ID	Requirement	Priority	Verification
FR-PY-01	The system SHALL generate executable Python from the validated flow representation in the principal pathway.	Must	End-to-end test
FR-PY-02	The Python generator SHALL NOT bypass the flow representation by recompiling directly from the original or processed pseudocode.	Must	Implementation review
FR-PY-03	Python generation SHALL be deterministic for a fixed validated graph and software version.	Must	Repeatability test
FR-PY-04	The system SHALL retain generated Python as an inspectable artefact.	Must	Artefact inspection
FR-PY-05	The system SHALL distinguish Python generation/compilation failures from behavioural test failures.	Must	Failure-localisation test
FR-PY-06	The system SHOULD preserve source to stage metadata sufficient to trace generated Python behaviour back to preceding flow and AST artefacts.	Should	Traceability inspection

Behavioural Testing and Test Categorisation

ID	Requirement	Priority	Verification
FR-TEST-01	The system SHALL execute generated Python against the frozen Common Tests package associated with the study configuration.	Must	Integration test
FR-TEST-02	The v4.37 study test package SHALL represent 50 algorithm suites and 156 declared test cases.	Must	Manifest/test discovery
FR-TEST-03	The system SHALL distinguish 129 Core Behavioural tests, 8 Pre-Freeze Validation cases and 19 Semantic Boundary Probes in the v4.37 study configuration.	Must	Category audit
FR-TEST-04	Core Behavioural results SHALL form the principal execution based behavioural evaluation surface.	Must	Scoring audit
FR-TEST-05	Semantic Boundary Probe outcomes SHALL be reported separately and SHALL NOT be automatically merged into the Core Behavioural score.	Must	Scoring audit
FR-TEST-06	The system SHALL record declared, evaluable, passed and not evaluable test counts for each algorithm instance where applicable.	Must	Result inspection
FR-TEST-07	A not evaluable behavioural outcome SHALL remain distinguishable from an observed failing behavioural outcome.	Must	Result-state test

ID	Requirement	Priority	Verification
FR-TEST-08	Passing all supplied behavioural tests SHALL be reported as evidence for the tested contract and SHALL NOT be labelled formal proof of universal semantic equivalence.	Must	Reporting review
FR-TEST-09	Test definitions and category assignments used for comparative execution SHALL remain frozen within a study run.	Must	Version/configuration audit

Automated Scoring and Human Review

ID	Requirement	Priority	Verification
FR-SCORE-01	The system SHALL retain automated rubric assessment separately from Core Behavioural performance.	Must	Scoring output inspection
FR-SCORE-02	Human Review SHALL be stored as an independent evidence stream and SHALL not overwrite automated evidence.	Must	Data-model inspection
FR-SCORE-03	Human Review records SHALL support terminal numerical outcomes, explicit completed non scorable outcomes and unresolved/incomplete states.	Must	State-model test
FR-SCORE-04	The explicit terminal state HUMAN_REVIEW_COMPLETE_NOT_EVALUABLE SHALL contribute 0% to the fixed-denominator Human Review Overall.	Must	Formula test
FR-SCORE-05	A missing Human Review percentage without the explicit completed-non-scorable terminal state SHALL NOT be treated as zero.	Must	Negative formula test
FR-SCORE-06	Human Review Overall SHALL be withheld while unresolved Human Review records remain.	Must	Eligibility test
FR-SCORE-07	For the 50 algorithm study profile, Human Review Overall SHALL require 50/50 terminal Human Review outcomes.	Must	Eligibility test
FR-SCORE-08	The system SHALL expose the numerical only Human Review mean separately from the fixed denominator Human Review Overall.	Must	Dashboard/report inspection
FR-SCORE-09	Automated Overall SHALL be calculated as the arithmetic mean of Mean Semantic Behavioural Score and Aggregate Common Test Accuracy.	Must	Formula test
FR-SCORE-10	Combined Final Score SHALL be calculated as the arithmetic mean of Automated Overall and eligible Human Review Overall.	Must	Formula test
FR-SCORE-11	Cross agent Combined Final ranking SHALL be produced only when every compared agent has an eligible Human Review Overall; otherwise ranking SHALL remain automated only and provisional.	Must	Ranking-state test
FR-SCORE-12	Scoring outputs SHALL retain enough component values to explain how every aggregate score was derived.	Must	Audit inspection

Artefact Retention, Audit and Traceability

ID	Requirement	Priority	Verification
FR-ART-01	For each model algorithm instance, the system SHALL retain the original pseudocode and processed pseudocode.	Must	Artefact inspection
FR-ART-02	The system SHALL retain parser/processing reports and machine readable AST evidence where generated.	Must	Artefact inspection
FR-ART-03	The system SHALL retain machine readable flow evidence and generated visual flow artefacts where generated.	Must	Artefact inspection
FR-ART-04	The system SHALL retain generated Python, compilation information and behavioural outcomes where available.	Must	Artefact inspection
FR-ART-05	The system SHALL retain study-scoring and Human Review artefacts where applicable.	Must	Artefact inspection
FR-ART-06	The system SHALL support reverse traceability from a study level result to the relevant algorithm instance and its preceding artefacts.	Must	Traceability test

ID	Requirement	Priority	Verification
FR-ART-07	Failure reporting SHALL identify the earliest observable stage at which processing could not continue or semantic/behavioural disagreement became evident where this can be determined.	Must	Failure localisation review
FR-ART-08	Study packages SHOULD support integrity verification through manifest/checksum evidence for retained files.	Should	Checksum verification

Trusted Baseline and Study Readiness

ID	Requirement	Priority	Verification
FR-VAL-01	Before comparative model evaluation, the system SHALL process a trusted corresponding 50 algorithm baseline through the frozen infrastructure.	Must	Baseline run
FR-VAL-02	The trusted baseline SHALL be used as a study readiness control and SHALL NOT act as a runtime oracle requiring model outputs to match one reference implementation.	Must	Method/configuration review
FR-VAL-03	A trusted-baseline processing failure SHALL trigger investigation and revalidation before model specific failure is interpreted experimentally.	Must	Readiness procedure review
FR-VAL-04	The system SHALL retain baseline validation outputs separately from experimental model outputs.	Must	Artefact inspection
FR-VAL-05	Pipeline changes intended for study use SHALL pass regression validation before being adopted into a frozen experimental configuration.	Must	Regression evidence
FR-VAL-06	Once the comparative experimental run begins, parser, canonicalisation, compiler, test and scoring behaviour SHALL remain version stable for all compared datasets.	Must	Version audit

Dashboards and Analytical Reporting

ID	Requirement	Priority	Verification
FR-DASH-01	The system SHALL provide a comparative heat map view of algorithm level model outcomes.	Must	Dashboard inspection
FR-DASH-02	The system SHALL provide model level radial comparison of reference categories and pipeline outcomes.	Must	Dashboard inspection
FR-DASH-03	The system SHALL provide per-model 50 algorithm radial profiles showing individual behavioural scores.	Must	Dashboard inspection
FR-DASH-04	The system SHALL provide a Human Review dashboard showing completion, reviewer, criterion level and semantic issue summaries.	Must	Dashboard inspection
FR-DASH-05	The system SHALL provide a Final Scoring dashboard exposing automated, Human Review and combined scoring components.	Must	Dashboard inspection
FR-DASH-06	Analytical dashboards SHALL operate as read only views over saved study artefacts and SHALL NOT participate in adjudication or alter authoritative raw scores.	Must	Implementation review
FR-DASH-07	The Final Scoring dashboard SHALL support export of the final scoring table as CSV.	Must	Export test
FR-DASH-08	Dashboard derived model summaries SHALL be data driven rather than hard coded to an expected ranking or named winning model.	Must	Implementation/data test

Versioning and Reproducibility

ID	Requirement	Priority	Verification
FR-VER-01	The pipeline SHALL expose a software/configuration version sufficient to identify the processing baseline used for a study run.	Must	Version inspection
FR-VER-02	Study artefacts SHALL record or preserve sufficient context to associate outputs with the frozen pipeline and test configuration.	Must	Metadata inspection

ID	Requirement	Priority	Verification
FR-VER-03	The system SHALL preserve deterministic downstream transformations wherever the input representation and configuration permit.	Must	Repeatability test
FR-VER-04	Changes after a frozen study release SHALL not retroactively alter retained v4.37 experimental results.	Must	Release/version audit
FR-VER-05	Post study corrections MAY be implemented in later releases provided the original study version and its results remain identifiable and reproducible.	May	Release audit
FR-VER-06	Repository documentation SHOULD identify the release/version associated with published study artefacts.	Should	Repository inspection

Non-Functional Requirements

ID	Quality attribute	Requirement	Priority
NFR-DET-01	Determinism	For identical accepted inputs, versioned configuration and deterministic downstream stages, the system SHALL produce repeatable structural outputs.	Must
NFR-SEM-01	Semantic integrity	The pipeline SHALL prefer explicit failure over semantic invention when meaning cannot be established safely.	Must
NFR-REP-01	Reproducibility	A study result SHOULD be reproducible from retained inputs, versioned pipeline artefacts, frozen tests and explicit scoring rules.	Must
NFR-AUD-01	Auditability	Every aggregate result SHALL be traceable to lower level evidence sufficient to explain its derivation.	Must
NFR-REL-01	Failure isolation	Stage failures SHALL be classified at the stage where they occur to minimise misattribution to downstream components.	Must
NFR-MNT-01	Maintainability	Transformation stages SHOULD remain modular so that parser, flow, generator, tests and dashboards can be validated independently.	Should
NFR-MNT-02	Regression safety	Changes to core transformation or scoring components SHALL be accompanied by regression testing appropriate to the affected stage.	Must
NFR-POR-01	Portability	Stored AST/flow/scoring artefacts SHOULD use machine readable, non proprietary or widely accessible formats where practical.	Should
NFR-POR-02	Diagram portability	Flow visualisation SHOULD remain exportable / importable through the supported Draw.io compatible representation.	Should
NFR-USA-01	Human inspectability	Original and transformed representations SHALL remain available in forms that permit human inspection without requiring reconstruction from aggregate scores.	Must
NFR-USA-02	Dashboard usability	Analytical dashboards SHOULD provide clear legends, labels and model/algorithm identifiers and remain usable on common desktop browser sizes.	Should
NFR-DAT-01	Data integrity	Study packages SHOULD support file integrity checking and detect missing or altered retained artefacts.	Should
NFR-SEC-01	Credential safety	API keys, access tokens and other secrets SHALL NOT be embedded in published study artefacts or repository documentation.	Must
NFR-PRI-01	Data minimisation	Published artefacts SHOULD contain only personal information necessary for research provenance; reviewer identifying fields SHOULD be removable when public release does not require them.	Should
NFR-EXT-01	Extensibility	The architecture SHOULD permit future alternative controlled intermediate representations to enter defined downstream stages without redefining the entire evaluation framework.	Should

Data Requirements

ID	Data object	Requirement
DR-01	Algorithm instance identity	Each record shall retain a model/dataset identifier and algorithm identifier.
DR-02	Input provenance	Original pseudocode and the associated natural language specification reference shall remain distinguishable from transformed artefacts.
DR-03	Canonicalisation evidence	Processed pseudocode, processing profile/version and processing report shall be retainable.
DR-04	Structural evidence	AST and flow representations shall be stored in machine readable form where the stage completes.
DR-05	Execution evidence	Generated Python, compilation/runtime status and behavioural test outcomes shall be stored where applicable.
DR-06	Test evidence	Declared/evaluable/passed/not evaluable counts and test category identity shall remain accessible for scoring and audit.
DR-07	Human Review evidence	Review status, rubric results, terminal state information, agreement data and justifications shall be stored separately from automated scores.
DR-08	Aggregate evidence	Model level summaries shall be derivable from algorithm level evidence rather than existing only as manually entered dashboard values.
DR-09	Version provenance	Study data shall be associable with the relevant pipeline release and Common Tests release.
DR-10	Integrity evidence	When a packaged study release is produced, checksum/manifest data should be available to verify retained files.

External Interfaces

ID	Interface	Requirement
IF-01	Model output interface	Text pseudocode conforming sufficiently to the supported strict dialect after permitted canonicalisation.
IF-02	Draw.io interface	Structured Draw.io compatible flow representation for export and alternative input import.
IF-03	Python runtime	Python 3 execution environment used for generated code and behavioural testing.
IF-04	Common Tests	Frozen versioned test package mapped to the configured algorithm set.
IF-05	Browser dashboard	HTML/JavaScript analytical views for comparative, radial, Human Review and final scoring outputs.
IF-06	CSV export	Final scoring dashboard export for downstream tabular analysis.
IF-07	Repository	GitHub project repository used for versioned distribution of source, documentation and supporting artefacts.

Study Specific Acceptance Baseline

The following values are not universal architectural limits. They are acceptance evidence associated with the frozen v4.37 study configuration and can be used to verify that a reproduced environment corresponds to the documented experimental baseline.

ID	Acceptance area	Criterion	v4.37 evidence
AC-01	Trusted baseline	All 50 trusted baseline algorithms process successfully through the defined readiness procedure.	50/50 successful; 100% mean calibration
AC-02	Common Tests package	Test discovery and categorisation match the frozen study package.	50 suites; 156 declared: 129 Core Behavioural, 8 Pre-Freeze Validation, 19 Semantic Boundary Probes
AC-03	Pytest regression	The v4.37 regression suite completes with the documented pass state.	1,377 passed; 3 skipped; 5 subtests recorded separately
AC-04	unittest regression	The v4.37 unittest suite completes with the documented pass state.	481 passed; 1 skipped
AC-05	Package integrity	Packaged study artefacts satisfy the documented checksum verification, subject to the known documented discrepancy.	1,087 files verified; known flow-implementation checksum discrepancy documented
AC-06	Principal-path demonstration	The representative summation example completes the pseudocode to execution pathway.	4/4 demonstration cases pass
AC-07	Alternative flow-input demonstration	The structured flow input Tic-Tac-Toe example imports and reaches executable testing.	35 shapes; 50 connectors; 2/2 demonstration cases pass
AC-08	Study dataset consistency	All experimental datasets expose the expected algorithm identity set.	6 datasets × 50 algorithms = 300 instances; no algorithm-ID mismatch
AC-09	Human Review completeness	Final combined ranking eligibility is achieved only after terminal Human Review coverage is complete.	50/50 terminal outcomes per model for the completed study

Requirements Traceability Summary

Area	Requirements	Primary evidence	Verification
Input / canonicalisation	FR-IN-*; FR-CAN-*	Original and processed pseudocode, processing report, profile/configuration	Unit tests, artefact inspection, reproducibility checks
Parser / AST	FR-AST-*	Parser report, AST machine artefact, AST visualisation	Parser regression and stage inspection
Flow representation	FR-FLOW-*	Machine graph, visual flow, Draw.io export	Graph validation, transformation tests
Alternative flow input	FR-ALT-*	Imported graph and downstream generated artefacts	Tic-Tac-Toe end-to-end demonstration
Python generation	FR-PY-*	Generated Python, compilation report	Generator regression and execution tests
Behavioural evaluation	FR-TEST-*	Common Test reports and category-labelled outcomes	Frozen Common Tests v1.4.2 study package
Scoring / Human Review	FR-SCORE-*	Rubric records, terminal states, final scoring data	Formula/eligibility tests and dashboard inspection
Traceability / audit	FR-ART-*	Per-instance retained artefact tree and reports	Reverse trace inspection and checksum validation
Readiness / validation	FR-VAL-*	Trusted baseline, regression reports, release evidence	Study-readiness gate and validation suite
Dashboards	FR-DASH-*	Comparative, radial, Human Review and final scoring HTML	Read-only dashboard inspection and CSV export
Versioning	FR-VER-*	Version identifiers, frozen configuration, release artefacts	Release/configuration audit

Out of Scope

- Training, fine tuning or modifying the internal parameters of any transformer based language model.
- Automatically repairing an algorithm whose required logic is absent or incorrect in the model generated response.
- Treating the trusted baseline as the only valid algorithmic implementation or requiring structural similarity to it.
- Formal verification or proof of semantic equivalence for all possible program inputs.
- Using graphical node placement, colour or drawing aesthetics as execution semantics.
- Claiming the current strict pseudocode dialect to be a universal standard or validated general purpose Controlled Intermediate Language.
- Using dashboard interaction to modify study evidence, adjudication or frozen raw scores.

Future Extension Considerations

The current requirements intentionally preserve a boundary between the implemented v4.37 system and future research. Potential extensions include testing a model independent Controlled Intermediate Language, comparing textual and graphical controlled representations, adding additional executable target languages, broadening the algorithm corpus and introducing further structural or behavioural analysis. Such extensions should be introduced as new versioned requirements rather than retroactively redefining the frozen study baseline.

Repository and Configuration Management

The project repository provides the designated public location for the NL-FDEP source code, documentation and supporting materials. Releases used for published experimental results should be tagged or otherwise version identified so that later changes do not obscure the configuration used for the reported study.

Repository: https://github.com/ajohnson-2210/Final_Project

Reference Basis

Source	Role in this specification
Cambridge Assessment International Education (2023)	Cambridge International AS & A Level Computer Science 9618: Pseudocode Guide for Teachers. Version 1. Used as the established basis from which the NL-FDEP strict pseudocode conventions were adapted.
NL-FDEP v4.37 study artefacts	Architecture, methodology, validation package, Common Tests evidence, Human Review records and analytical dashboards supplied with the completed study.
Project repository	ajohnson-2210/Final_Project, GitHub. Repository location for project source and documentation.

Cambridge guide: https://www.cambridgeinternational.org/Images/697401-2026-pseudocode-guide-for-teachers.pdf

Approval and Change Control

This specification represents the documented as built requirements of the v4.37 study baseline. Any change that alters parsing rules, canonicalisation behaviour, graph semantics, Python generation, behavioural tests, scoring formulas or Human Review eligibility shall be versioned and revalidated before being treated as equivalent to this baseline.